COMP 4300 Assignment 2:
Go-Back-N Automatic Repeat Request Over UDP

Keenan Gowdar - 7861237
November 5th. 2026

Compilation and Execution

The /assignment2 directory has all the files necessary for ns-3 to build the project. Ensure it is dropped inside the /scratch directory inside of a completed ns-3 installation.

Once /scratch/assignment2 has been created, open a terminal and navigate to the ns-3 root dir and run (assuming you have built ns3):

```
$ ./ns3 run "scratch/assignment2/main --dataLoss=0.0 --ackLoss=0.0"
```

- dataLoss and ackLoss represent the probabilities associated with DATA/ACK loss, respectively, between 0 and 1.

Implementation Design (Overview of Classes and Functions)

main.cc

- Defines the network topology and configuration parameters.
- Serves as the main entry point into the simulation, creating the nodes, p2p link, application classes and printing the associated simulation statistics.

gbnReceiver.cc

- Receives DATA packets over UDP
- Tracks the next expected sequence number
- Discards duplicate / out-of-order DATA
- Sends a cumulative ACK after EVERY received DATA packet

StartApplication() is called upon simulation start, which sets the event trigger for a HandleData() call dependent on when the shared socket gets data.
HandleData() validates incoming DATA, updates the expected sequence number, and calls SendAck() to acknowledge the last in-order packet.

gbnSender.cc

- Maintains SlidingWindow state
- Transmits DATA packets containing a 4-byte SEQ and 1000-byte payload
- Processes cumulative ACKs
- Uses ONE retransmission timer for the oldest outstanding DATA
- On timeout, retransmits ALL unacknowledged DATA (Go-Back-N)

SendAvailable() fills the sliding window by transmitting new DATA packets through SendData().
HandleAck() processes cumulative acknowledgments, advances the window, and

manages the retransmission timer.

HandleTimeout() retransmits all outstanding DATA packets and restarts the timer.

slidingWindow.cc

- Handles base, nextSeqNum, and outstanding ACK state.

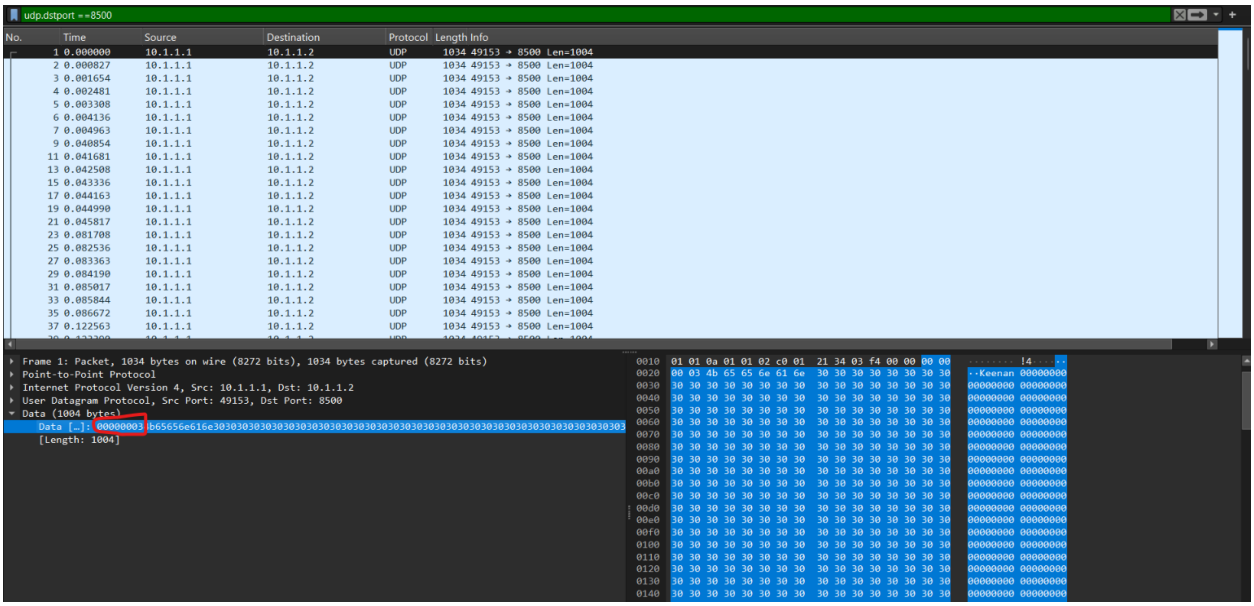
GetNextPacket() advances the next sequence number when a new packet is sent.
Acknowledge() processes cumulative ACKs and advances the window base.
GetOutstandingSequence() identifies packets awaiting acknowledgment for retransmission.

packetEncoding.cc

- Encodes/Decodes packets in network byte order.

EncodeSequenceNumber() and DecodeSequenceNumber() convert between 32-bit integers and their 4-byte big-endian representations.

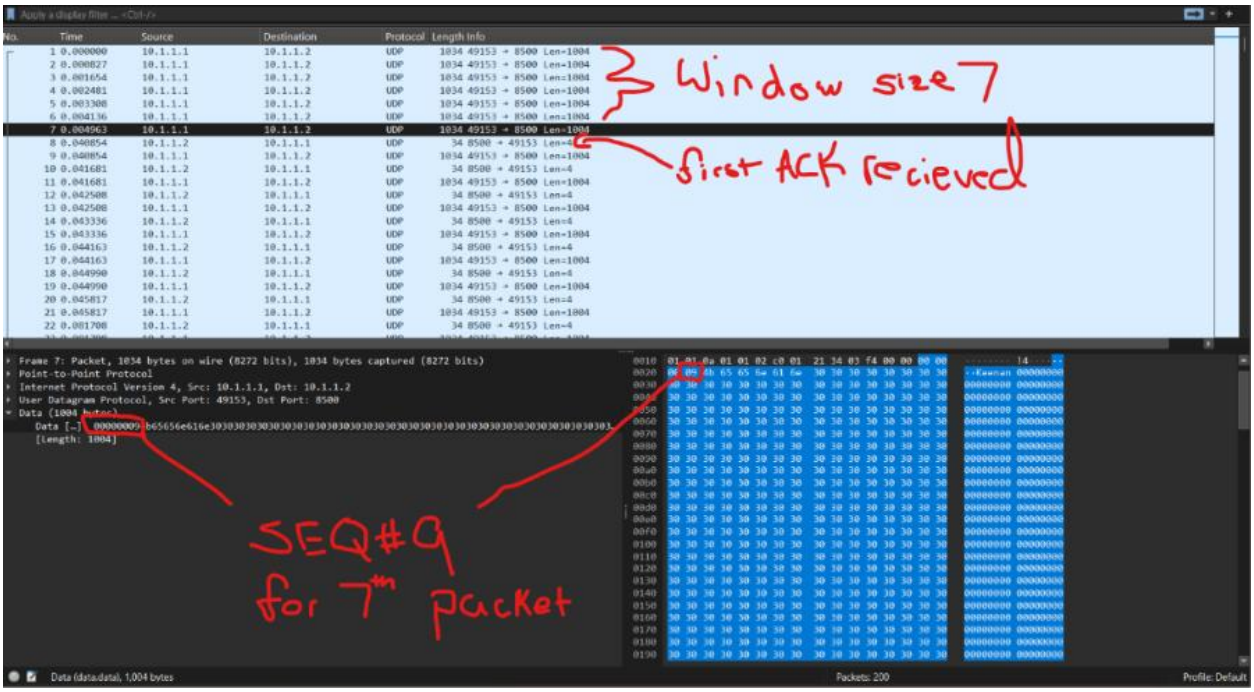
Demonstrations



The above pcap file (opened in WireShark) illustrates the senders packets (10.1.1.1) overport 8500. Here we can see the first packet sent starts with sequence number 3, which is the second most right digit in my student number (781237). Also note the length of 1004, with the first four bytes designating the sequence number. It wasn't possible to include the

entire payload on the screen, but as you can see, the payload contains my name “Keenan”, followed by zeroes, with a length of 1004.

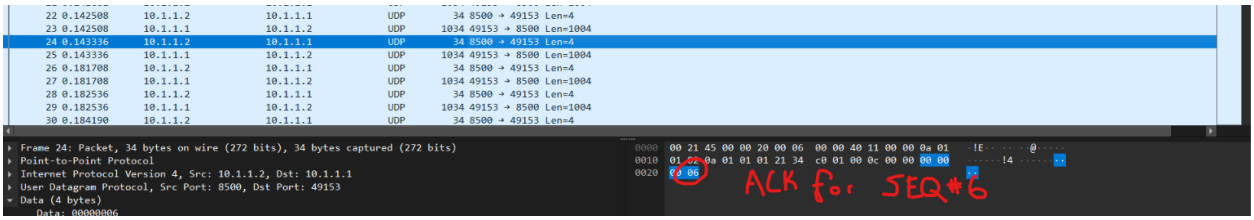
Also note the Network Byte Order.

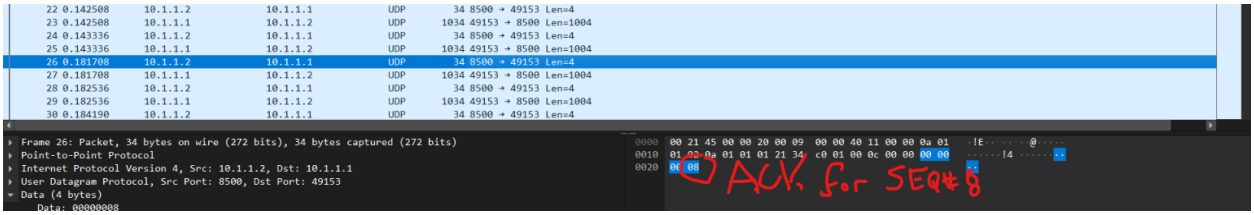


Now, without the udp.dstport == 8500 filter, we note that our window size is 7. This is demonstrated by the first 7 packets being sent out before the 1st ACK comes back. Notice the SEQ #9 for the 7th packet as highlighted above.

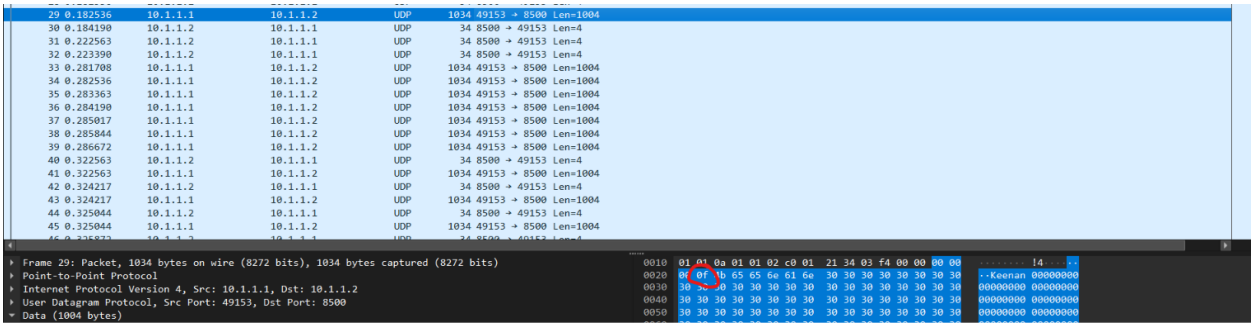
Cumulative ACK and retransmission:

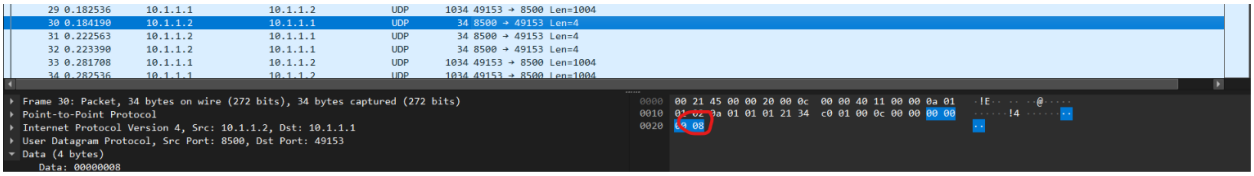
We note the following cumulative ACK behaviour, as the ACK for SEQ#7 is skipped due to ACK#8 being received prior to it.

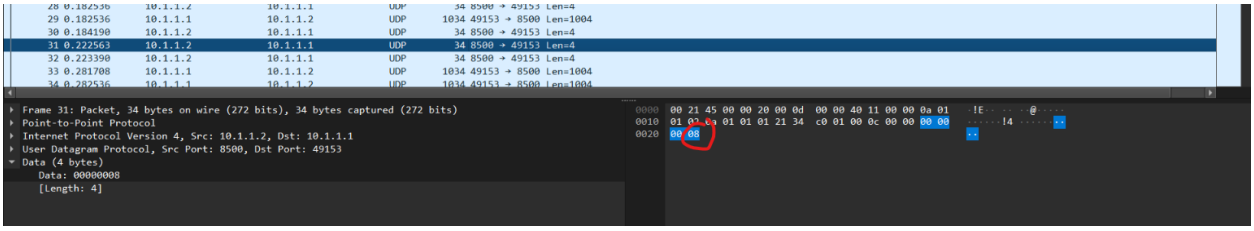


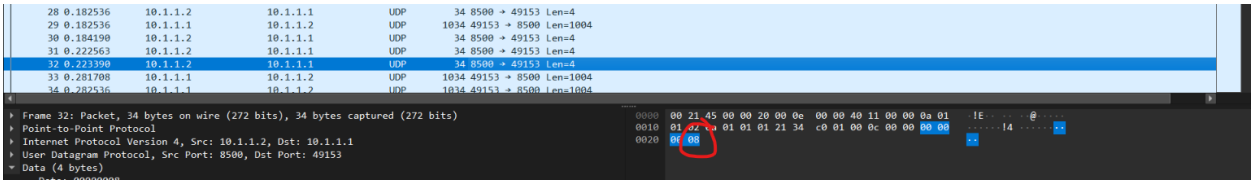


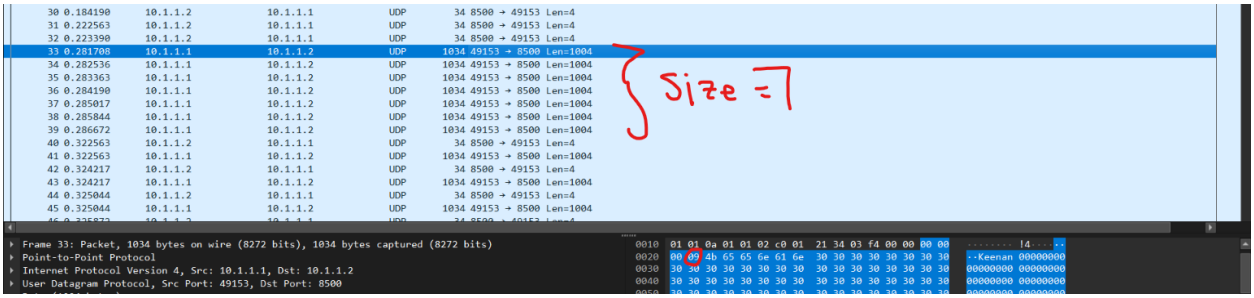
Now for retransmission, note the three ACK's for SEQ#8 are received by the sender AFTER the previous SEQ#15 was sent out, thus the sender retransmits the window of size 7, starting at SEQ#9



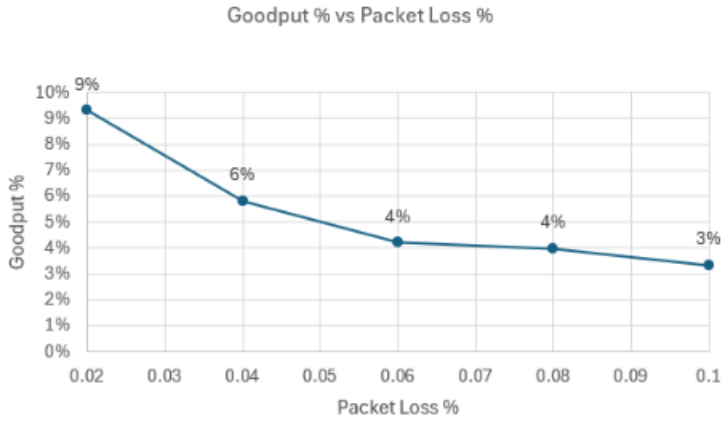








Performance Evaluation and Analysis



As we can see by the graph shown (and as expected), as the probability of packet loss increases, we see goodput decreasing. This relationship seems to resemble exponential decay; after we first introduce some probability of packet loss, we notice a huge decrease in goodput percentage, with increases to this probability having a diminishing impact on goodput percentage.

But why does introducing a small bit of packet loss have such a huge effect on goodput percentage?

My initial explanation revolves around the inefficiencies introduced by Go-Back-N's retransmission mechanism. When a DATA packet is lost, the receiver discards subsequently received out-of-order packets, even if they arrive successfully. Consequently, when the sender's retransmission timer expires, it may need to retransmit multiple packets, not just the one originally lost.

These retransmissions consume network bandwidth without increasing the amount of unique application data successfully delivered. When the sender's window becomes full, it may be forced to remain idle until acknowledgments arrive, or a retransmission timeout occurs. This additional delay increases the total time required to deliver the original 100 packets, directly reducing goodput.